Open in app ↗ Sign up Sign in

Medium Search

Glassdoor Enginee... · Follow publication

Featured

Improving Embedding-Based Candidate Generation for Recommender Systems with a Two-Tower Model

9 min read · Mar 18, 2026

Ahmad Khan Follow

Listen Share

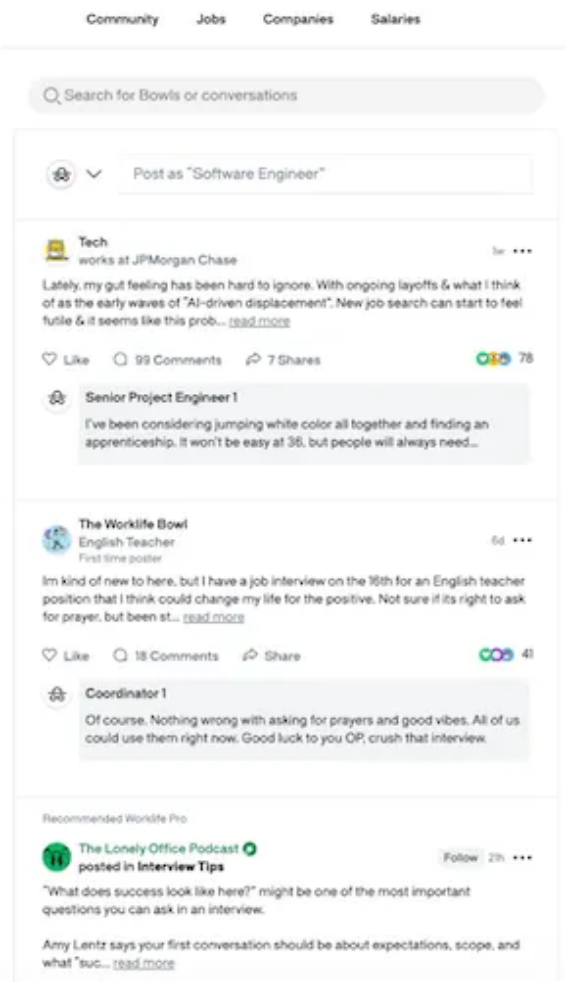
Overview

At Glassdoor (part of Indeed), millions of professionals come to learn about workplaces, ask career questions, and share experiences through our Community platform. When users open the Community Home Feed, they expect to see posts that are relevant to their interests, industry, and career stage.

However, generating that feed is not a simple task.

At any moment, thousands of new posts across hundreds of community feeds compete for attention. From discussions about interview experiences to workplace culture questions and industry advice, the platform contains a diverse and constantly evolving stream of content. The challenge is to quickly identify which of these posts are most relevant for each user.

To address this problem, we built a **multi-stage machine learning recommendation system** that efficiently sources, filters, and ranks posts before they appear in the user's feed. In a previous post, Inside Glassdoor's Multi-Stage Recommendation System, we described the architecture of this system and how the different stages work together to deliver personalized content.

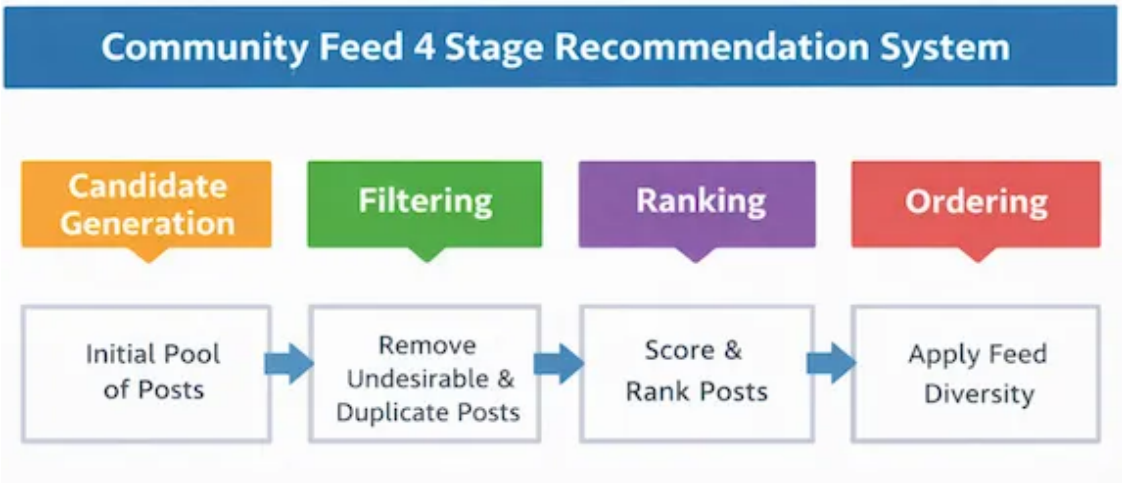


Snapshot of the Glassdoor Community Home Feed

At a high level, the system consists of four stages:

1. **Candidate Generation** — retrieves a large set of potentially relevant posts
2. **Filtering** — removes posts that are undesirable, duplicated, or already seen
3. **Ranking** — predicts which posts a user is most likely to engage with
4. **Ordering** — applies heuristics to maintain diversity and freshness in the feed

Each stage progressively narrows the set of posts until only the most relevant posts remain.



4 Stage ML RecSys Design

In this blog post, we focus on the **Candidate Generation stage**, which is responsible for retrieving an initial pool of potentially relevant posts from a very large pool of posts across many different feeds.

For Candidate Generation, we developed a **Two-Tower Neural Network Model** that learns embeddings for users and posts based on real interaction data.

In the rest of this blog post, we will walk through:

- how the Two-Tower architecture works
- how we train embeddings using user-post interactions
- improvements to model training such as mixed negative sampling and self-supervised learning
- and how this model improved engagement on the Glassdoor Community feed

Embedding Based Candidate Generation

Candidate generation is a large search and retrieval problem. Given thousands (or millions) of posts, the goal is to find a small subset of posts that could be relevant to a user. Additionally, we want to ensure:

1. **High recall** — ensure that relevant posts are included in the candidate set so later ranking stages can select the best ones

2. **Low latency** — retrieve candidates quickly enough to support real-time feed generation

For problems requiring both high recall and low latency, **Approximate Nearest Neighbor (ANN)** embedding search is a practical solution. We embed both users and posts into a vector database and perform ANN retrieval to efficiently find posts whose embeddings are most similar to a given user.

There are many algorithms that can generate such embeddings for ANN. At Glassdoor in the past we have experimented with pre-trained language models. Our first iteration simply used the GloVe language model. Later, we used the more powerful Sentence Transformer model to embed users and posts based on the content they interacted with. But these embedding methods used pre-trained, out of the box, models not fine-tuned on actual user-post interactions and mostly calculated semantic similarity for user and posts based on text similarity.

To address these limitations, we moved to an enhanced approach last year using a trained **Two Tower Neural Network Model** where we train the user and post embeddings based on the user-post interaction prediction task itself.

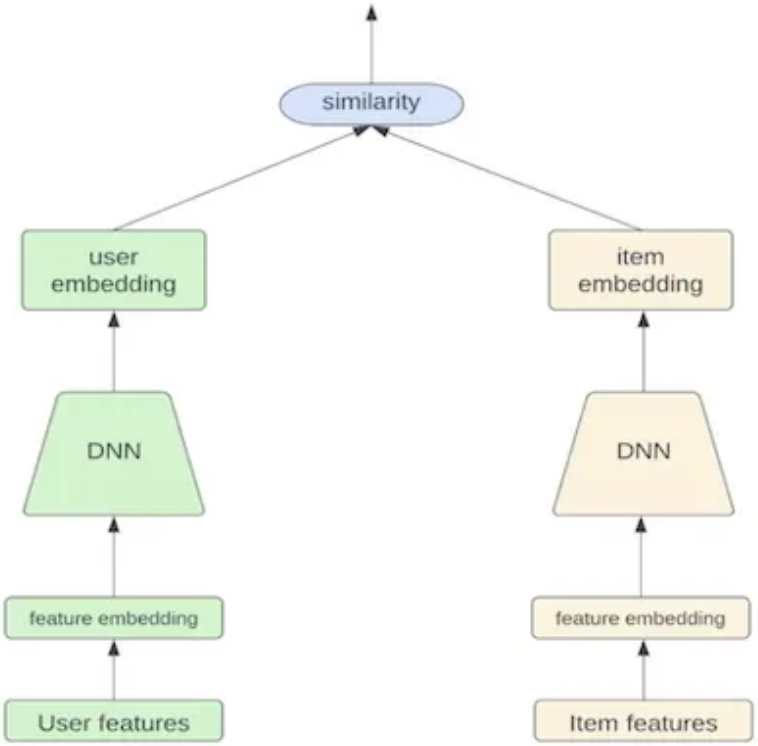
Two-Tower Model for ANN-Based Candidate Generation

In a multi-stage recommender system, **Two-Tower models** are commonly used in industry in the **Candidate Generation stage** to generate embeddings.

A Two-Tower model consists of:

- a **User Tower**, which embeds a user's profile and platform engagement using computed and other metadata user features
- a **Post Tower**, which embeds posts using post features into the same embedding space

Each tower is a neural network model, in our case a stack of linear layers followed by a ReLU Activation, as shown below.



Two Tower Neural Network Model Architecture

Both towers are trained jointly so that users are close in the vector embedding space to posts they are likely to interact with and far from those they are less likely to interact with. For our Two Tower model, we utilize the following user and post features to generate the embeddings.

User Tower Features	Post Tower Features
User Profile Metadata (e.g: employer, job title, location, industry, submitted resume)	Post Author metadata (e.g: job title, employer, location)
User's Aggregate Embedding of Recently Interacted Posts	Post Metadata and Topics (e.g: NER entities, Industry, Inferred Topics)
User's Recent Platform Engagement Signals (interactions with jobs, reviews, salaries)	Post Engagement Signals (post clicks, reactions, comments)
User's Recent Community Engagement Signals (e.g: number of user post clicks, post reactions, post comments last 30d etc)	Post Comment Embeddings
User's Subscribed Feeds	Post Feed Name

User and Post Tower Input Features for Two Tower Model

For text-based fields (like post text, comment text, topics), we generate semantic embeddings using a pre-trained Sentence Transformer model. These embeddings are then concatenated with structured features and used as inputs to the Two-Tower neural network.

Two Tower Model Training

To learn meaningful user and post embeddings, two-tower models are typically trained using **Contrastive Loss or Noise Contrastive Estimation** style objectives, often implemented as **sampled softmax** over **in-batch negatives.**

Get Ahmad Khan's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

Remember me for faster sign in

In our case the core idea was to transform recommendation into a **binary**

discrimination problem: given a user embedding, the model must assign a higher similarity score (e.g: dot product) to a *true (positive) post interaction's embedding* than to a set of *negative (noise) posts embedding* sampled from the post corpus.

Specifically, for each training example:

- A **positive post** is drawn using the user's historical post interactions. These are posts the user actually interacted with (i.e: clicked, reacted) previously in our training data.
- Multiple **negative posts** are sampled. These are posts the user did not interact with in our training data.

The model computes similarity scores (dot products) between the user embedding and both positive and negative post embeddings. A contrastive style loss then encourages the positive user-post pair to score higher than the negative user-post pairs.

$$\mathcal{L} = -\log \frac{\exp(u \cdot v^+)}{\exp(u \cdot v^+) + \sum_{i=1}^{N} \exp(u \cdot v_i^-)}$$

Two Tower Training Loss Objective

This objective approximates a full softmax over all posts — which would be computationally infeasible at scale — while still learning an embedding space where relevant user–post pairs are close and irrelevant pairs are far apart.

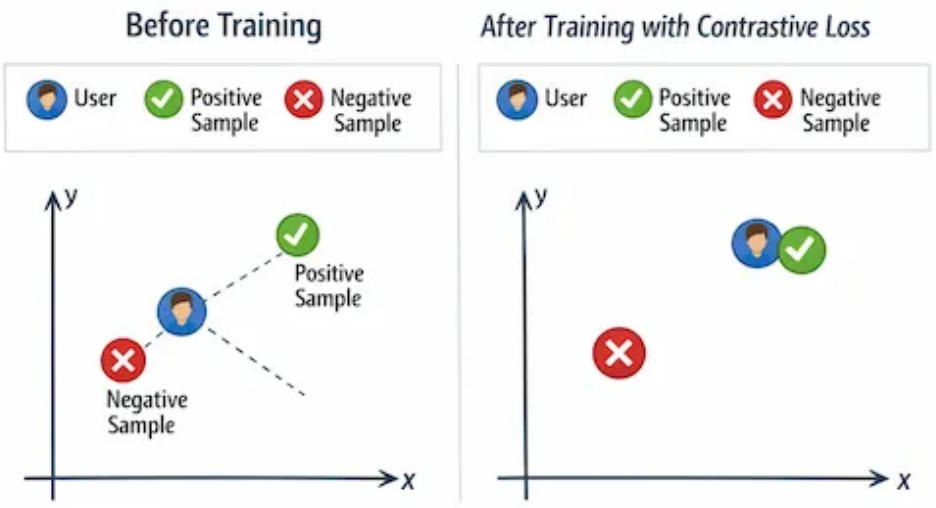


Illustration of changes in user and positive/negative post embeddings after model training

Two Tower Model Training Optimizations

One of the key challenges in training two-tower models is **choosing which negative posts to show the model**. If all negatives are random or trivial, the model quickly learns to separate "obvious" non-relevant posts, but struggles with more subtle distinctions. If all negatives are other posts in the batch, then the model only learns embeddings for popular posts more likely to show up in batches of user-post interactions while ignoring the long tail of less popular posts entirely. This is where **Mixed Negative Sampling** comes in, an idea described in the Google paper Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations which we also use for training the Two Tower model.

Mixed Negative Sampling combines two different types of negative streams when training the model.

1. **In-Batch Negatives:** Other posts in the same training batch act as negatives for a given user. This is efficient and can be done via just matrix multiplication during batch training.

2. **Random Negatives:** Posts sampled uniformly from the entire post corpus.

By mixing negatives from these two sources, the model learns a richer embedding space for both popular and less popular posts.

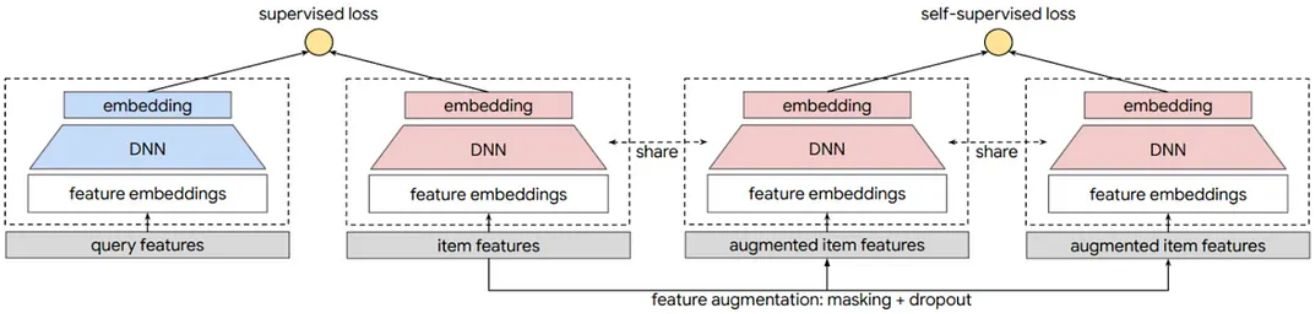
Another improvement to the Two Tower model training we implemented is adding a **Self-Supervised Learning (SSL) loss** alongside the main contrastive loss objective. This helps the model learn richer embeddings that capture intrinsic structure in user and post behavior, not just interactions. The approach is inspired by work from another Google paper *"Self-supervised Learning for Large-scale Item Recommendations"*.

The idea is to create **augmented views** of users or posts by masking part of the user and post's input feature vector with zero values. These augmentations generate multiple "views" of the same underlying entity's input feature vector. The model is also trained on an additional auxiliary contrastive task where positives and negatives are defined as follows.

- *Positive pairs*: Two augmented feature vector views from the same underlying users (or posts)
- *Negative pairs*: Two augmented feature vector views from different underlying users (or posts)

Formally, the SSL loss encourages embeddings from the same entity to be **close in the latent space**, while embedding views from different originating sources remain far apart. Combined with the standard contrastive loss, this gives the model two complementary learning signals:

1. **Supervised contrastive task (main contrastive loss)** — teaches the model which posts a user is likely to interact with
2. **Self-supervised task** — teaches the model the underlying structure of user behavior and posts independent of interactions



Source: "Self-supervised Learning for Large-scale Item Recommendations"

This helps to improve generalization to rare or long-tail posts which are never seen during training using just the supervised contrastive task and also improves embeddings for sparsely engaged users who don't have much platform engagement history. It also captures similarity between users and posts beyond just explicit interactions.

Model Evaluation and AB Test Results

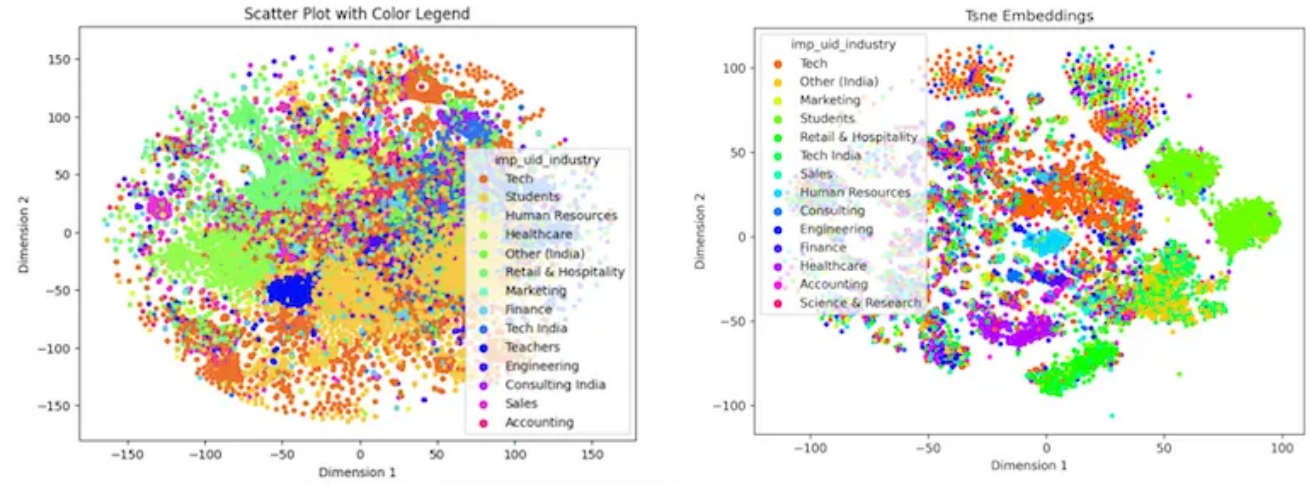
The new Two Tower model was trained with 2 months of our historical user-post community interaction data and tested offline on the following one week of separate test data. Offline results showed the model improving offline retrieval metrics such as Precision@K, Recall@K, F1@K, HitRate@K by 40–60% relative to our previous Sentence Transformer embedding baseline.

We then validated the model's impact through a series of A/B tests. Cumulatively, the model drove a substantial improvement in user community engagement, measured by a metric we define internally: **eCRAUs** (Engaged Community Registered Active Users), which counts users who took *any* engagement action (including clicks, reactions, or comments).

- **+5%** cumulative, statistically significant, improvement in the total number of **eCRAUs**
- **+5%** cumulative, statistically significant, improvement in **Clicks per User**, while post reactions and comments increased by more than 10%, though not statistically significant

Beyond direct engagement metrics, the model also delivered a noticeable improvement in our **feed diversification.** Users were shown post impressions from a wider variety of less common feeds, rather than the feed being dominated only by the biggest or most popular ones. These positive, statistically significant, results led to the adoption of the Two Tower model as part of our 4 Stage ML RecSys in production.

We further assessed the quality of the new Two Tower embeddings with embedding clustering analysis. We reduced the high-dimensional embeddings to 2D (using T-SNE) for visualization, and compared the new Two Tower embeddings against our old baseline using just Sentence Transformer.



(Left) Previous Sentence Transformer Embedding Clusters vs (Right) New Two Tower Embedding Clusters

Clustering Analysis Summary	Old Embeddings (Sentence Transformer)	New Embeddings (Two Tower Model)
Visualization	Represented as a large, undifferentiated 'blob'.	Created more distinct and separated clusters.
Industry Separation	Not a clear distinction based on the color legend (indicating user industry).	Clearer separation, with clusters loosely aligned with industry categories
Evaluation	Embeddings lack the ability to distinguish unique post/user attributes well.	Embeddings appear to better separate unique aspects of the post or users

Visual Analysis Summary of Embedding Clusters

This qualitative analysis was also supported by quantitative evaluations of the embeddings using clustering quality metrics like **Silhouette Score** and the **Calinski-Harabasz Index**, which also showed major improvement.

Model Deployment

The Two Tower model was deployed in production using MLflow and the model was served through a REST endpoint used by our Community Home Feed Recommendation System Service for inference. During inference, the client service sends the user ID and user features or post ID and post features and the model service returns the computed Two Tower based embeddings using the deployed model for users and posts. We precomputed these embeddings offline in batches and indexed them in our internal OpenSearch vector database for fast embedding retrieval using ANN search for real-time inference when the user opened their home feed.

More information around our ML Engineering and Experimentation architecture that we built for the Two-Tower model will be shared in a follow-up blog post.

Stay tuned for more!

References

1. Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations

2. Self-supervised Learning for Large-scale Item Recommendations

Machine Learning | Recommender Systems | Two Tower Model | Self Supervised Learning | AI

Follow

Published in Glassdoor Engineering Blog

339 followers · Last published Mar 18, 2026

The official blog of Glassdoor's engineering team, sharing ideas and learnings found during our mission of helping people everywhere find a job and company they love.

Follow

Written by Ahmad Khan

24 followers · 2 following

Staff Machine Learning Scientist @ Indeed

No responses yet

Write a response

What are your thoughts?